

Lucas Augusto da Silva Cardoso (202200332300)

DCA3703 | Programação Paralela | Relato de experimento

TAREFA 5: DESAFIOS DE SINCRONIZAÇÃO E DESEMPENHO NA BUSCA POR PRIMOS

Este documento detalha a abordagem metodológica e a solução implementada para o Experimento Computacional proposto na disciplina de Programação Paralela, em 25/08/2026.

1. EXPERIMENTO

"Implemente um programa em C que conte quantos números primos existem entre 2 e um valor máximo "a". depois, paralelize o laço principal usando a diretiva pragma omp parallel for sem alterar a lógica original. compare o tempo de execução e os resultados das versões sequencial e paralela. observe possíveis diferenças no resultado e no desempenho, e reflita sobre os desafios iniciais da programação paralela, como correção e distribuição de carga."

2. CONCEITOS RELEVANTES

- **Condição de corrida:** ocorre na programação paralela quando duas ou mais "threads" tentam acessar e modificar uma mesma variável compartilhada simultaneamente, sem nenhum mecanismo de controle de concorrência ou sincronização. Como o processador e o sistema operacional escalonam as tarefas de forma assíncrona e independente, a ordem exata de leitura e escrita na memória torna-se imprevisível. O resultado final do processamento passa, então, a depender de qual *thread* "vence a corrida" para gravar a instrução por último.

- **Balanceamento de carga:** refere-se à técnica de distribuir o trabalho computacional de maneira uniforme e justa entre todas as threads ou núcleos de processamento disponíveis no sistema. O objetivo principal é garantir que nenhuma unidade de execução fique ociosa enquanto outras continuam sobrecarregadas, minimizando o tempo total de espera.

3. CÓDIGO

Para a realização dos testes do experimento, foram desenvolvidos dois programas distintos em linguagem C, sendo uma versão estritamente sequencial e uma versão paralelizada. O objetivo dessa divisão é isolar a introdução do paralelismo e facilitar a comparação direta entre os códigos.

A lógica matemática de verificação de primalidade permaneceu intacta em ambos, concentrando as diferenças exclusivamente na infraestrutura das “threads” e nos métodos de medição de desempenho.

Quanto a medição de tempo, enquanto o código sequencial depende de bibliotecas nativas de tempo e utiliza chamadas de sistema do Linux para registrar a duração da execução, a versão paralela simplifica essa etapa utilizando a função “omp_get_wtime()” fornecida pela biblioteca do OpenMP. A alteração estrutural primária ocorre com a adição da diretiva “#pragma omp parallel for” antes do laço principal do segundo programa, responsável por distribuir as iterações de forma automática. Como a variável acumuladora é incrementada diretamente pelas tarefas simultâneas sem nenhum mecanismo de proteção de concorrência, essa paralelização ingênua introduz uma condição de corrida (“race condition”) na contagem dos resultados.

Por fim, a manipulação do programa paralelo difere da versão sequencial por exigir a “flag” -fopenmp durante a compilação no GCC, além da utilização do comando “export OMP_NUM_THREADS” no terminal para definir explicitamente a quantidade de *threads* ativas durante cada teste de execução.

Figura 1: Código do programa sequencial

```
#define _POSIX_C_SOURCE 199309L
#include <stdio.h>
```

```c
#include <stdlib.h>
#include <math.h>
#include <time.h>

// Função para medir o tempo de relógio absoluto (em segundos)
double recuperar_tempo_instantaneo() {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (double)ts.tv_sec + (double)ts.tv_nsec * 1e-9;
}

// Verifica se um número n é primo
int eh_primo(int n) {
    if (n <= 1) return 0;
    if (n == 2) return 1;
    if (n % 2 == 0) return 0;

    for (int i = 3; i < n; i += 1) {
        if (n % i == 0) return 0;
    }

    return 1;
}

int main(int argc, char *argv[]) {
    int a = 500000; // Valor máximo padrão
    if (argc > 1) {
        a = atoi(argv[1]);
        if (a < 2) {
            fprintf(stderr, "Erro: O valor de 'a' deve ser no mínimo 2.\n");
            return 1;
        }
    }

    printf("Calculando quantidade de primos entre 2 e %d...\n", a);

    int count = 0;
    double inicio = recuperar_tempo_instantaneo();
```

```
    for (int i = 2; i <= a; i++) {
        if (eh_primo(i)) {
            count++;
        }
    }

    double fim = recuperar_tempo_instantaneo();
    double tempo_total = fim - inicio;

    printf("Resultado: %d primos encontrados.\n", count);
    printf("Tempo de execucao: %f segundos\n", tempo_total);

    return 0;
}
```

Figura 2: Código do programa em paralelo

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>

// Verifica se um número n é primo
int eh_primo(int n) {
    if (n <= 1) return 0;
    if (n == 2) return 1;
    if (n % 2 == 0) return 0;

    for (int i = 3; i < n; i += 1) {
        if (n % i == 0) return 0;
    }

    return 1;
}

int main(int argc, char *argv[]) {
    int a = 500000; // Valor máximo padrão
```

```
    if (argc > 1) {
        a = atoi(argv[1]);
        if (a < 2) {
            fprintf(stderr, "Erro: O valor de 'a' deve ser no mínimo 2.\n");
            return 1;
        }
    }

    printf("Calculando quantidade de primos entre 2 e %d...\n", a);

    int count = 0;
    double inicio = omp_get_wtime();

    #pragma omp parallel for
    for (int i = 2; i <= a; i++) {
        if (eh_primo(i)) {
            count++;
        }
    }

    double fim = omp_get_wtime();
    double tempo_total = fim - inicio;

    printf("Resultado: %d primos encontrados.\n", count);
    printf("Tempo de execucao: %f segundos\n", tempo_total);

    return 0;
}
```

4. RESULTADOS

O resultado dos testes do programa gerado pelo código do teste de cálculo dos números primos podem ser vistos nas Tabela 1 e 2 e gráfico da Figura 3 abaixo.

Tabela 1: Testes feitos com código sequencial

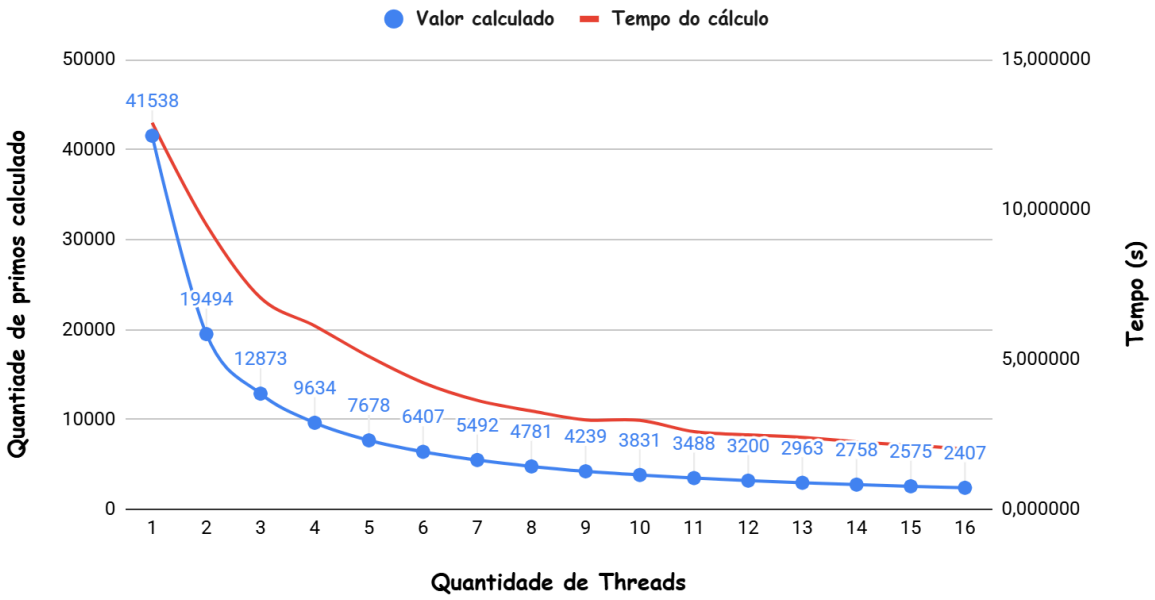
Valor calculado	Tempo (s)

41538	12,895257
41538	12,896946
41538	12,895502
41538	12,898406
41538	12,905388

Tabela 2: Testes feitos com código paralelo

Quantidade de threads	**Tempo (s)**	Valor calculado
1	12,898219	41538
2	9,496108	19494
3	7,060116	12873
4	6,123369	9634
5	5,100283	7678
6	4,226944	6407
7	3,637577	5492
8	3,282566	4781
9	2,98731	4239
10	2,97182	3831
11	2,59665	3488
12	2,48906	3200
13	2,40239	2963
14	2,25355	2758
15	2,13847	2575
16	2,01972	2407

Figura 3: Tempo e quantidade de primos cálculos em função da quantidade de “threads”



A análise dos dados exibidos acima revela um claro contraste entre os ganhos de velocidade obtidos pelo paralelismo e a grave falha na integridade dos resultados, decorrente da falta de mecanismos de sincronização.

O tempo de processamento apresenta uma redução consistente conforme o número de “threads” aumenta. A execução com uma única “thread” (que emula o comportamento sequencial) demandou 12,898219 segundos. Ao escalar a execução até o limite testado de 16 “threads”, o tempo despencou para apenas 2,01972 segundos.

Apesar da nítida aceleração, nota-se que o ganho de desempenho não é perfeitamente linear (por exemplo, passar de 1 para 2 “threads” reduziu o tempo de 12,89 segundos para 9,49 segundos, e não para a metade exata). Isso porque as “threads” responsáveis pelos blocos numéricos maiores ficam sobrecarregadas e ditam o tempo final de espera, limitando a eficiência ideal do paralelismo.

O fenômeno mais crítico e instrutivo do experimento, no entanto, é observado na coluna "Valor calculado". Na execução com 1 “thread”, o programa identifica corretamente o total matemático de 41.538 números primos. Contudo, assim que a concorrência é introduzida (2 “threads”), o valor contabilizado despenca quase pela metade, registrando apenas 19.494. A perda de dados se agrava severamente conforme mais núcleos são ativados, chegando ao alarmante

resultado de meros 2.407 primos identificados com 16 *threads*, uma perda de quase 95% da informação real.

Essa degradação vertiginosa é a comprovação empírica da condição de corrida (race condition). Como a diretiva “#pragma omp parallel for” foi inserida no código de forma ingênua, sem a utilização de uma cláusula de redução matemática ou de seções críticas para proteger a variável compartilhada, milhares de operações de incremento (“count++”) colidiram na memória. Quanto mais “threads” foram adicionadas ao processo, maior foi a frequência dessas colisões simultâneas. O resultado prático foi a sobreposição e o apagamento constante de dados, provando que otimizar o tempo de execução sem garantir o controle de concorrência invalida completamente o sistema.

5. CONSIDERAÇÕES FINAIS

O experimento evidenciou que a programação paralela impõe dois desafios centrais que vão além da simples aplicação de diretivas, como a correção dos dados e a distribuição de carga.

O obstáculo mais crítico é a garantia de resultados corretos. A ausência de proteção na variável compartilhada gerou condições de corrida que invalidaram completamente a saída de dados, provando que o ganho de velocidade é inútil se o sistema não for confiável.

O segundo desafio é a má distribuição de carga. Como verificar a quantidade de primos em intervalo de números altos exige muito mais esforço computacional do que testar números baixos, a divisão estática das tarefas sobrecarregou certas “threads” enquanto outras ficavam ociosas.

Conclui-se que o desenvolvimento paralelo exige uma mudança arquitetural no código. É preciso garantir que o acesso simultâneo à memória seja seguro e que o esforço de processamento seja balanceado de forma justa, extraindo o máximo do “hardware” sem comprometer a integridade do sistema.